SIAM J. OPTIM.
Vol. 16, No. 3, pp. 650–669
© 2006 Society for Industrial and Applied Mathematics

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

PRECISION CONTROL FOR GENERALIZED PATTERN SEARCH ALGORITHMS WITH ADAPTIVE PRECISION FUNCTION EVALUATIONS*

ELIJAH POLAK† AND MICHAEL WETTER‡

Abstract. In the literature on generalized pattern search algorithms, convergence to a stationary point of a once continuously differentiable cost function is established under the assumption that the cost function can be evaluated exactly. However, there is a large class of engineering problems where the numerical evaluation of the cost function involves the solution of systems of differential algebraic equations. Since the termination criteria of the numerical solvers often depend on the design parameters, computer code for solving these systems usually defines a numerical approximation to the cost function that is discontinuous with respect to the design parameters. Standard generalized pattern search algorithms have been applied heuristically to such problems, but no convergence properties have been stated.

In this paper we extend a class of generalized pattern search algorithms to include a subprocedure that adaptively controls the precision of the approximating cost functions. The numerical approximations to the cost function need not define a continuous function. Our algorithms can be used for solving linearly constrained problems with cost functions that are at least locally Lipschitz continuous.

Assuming that the cost function is smooth, we prove that our algorithms converge to a stationary point. Under the weaker assumption that the cost function is only locally Lipschitz continuous, we show that our algorithms converge to points at which the Clarke generalized directional derivatives are nonnegative in predefined directions.

An important feature of our adaptive precision scheme is the use of coarse approximations in the early iterations, with the approximation precision controlled by a test. We show by numerical experiments that such an approach leads to substantial time savings in minimizing computationally expensive functions.

Key words. algorithm implementation, approximations, generalized pattern search, Hooke–Jeeves, Clarke's generalized directional derivative, nonsmooth optimization

AMS subject classifications. 90C30, 90C56, 90C59, 65K05

DOI. 10.1137/040605527

1. Introduction.

Generalized pattern search (GPS) algorithms are derivative free methods for the minimization of smooth functions, possibly with linear inequality constraints. Examples of pattern search algorithms are the coordinate search algorithm [22], the pattern search algorithm of Hooke and Jeeves [14], and the multidirectional search algorithm of Dennis and Torczon [9]. What they all have in common is that they define the construction of a mesh, which is then explored according to some rule, and if no decrease in cost is obtained on mesh points around the current iterate, then the mesh is refined and the process is repeated.

*Received by the editors March 23, 2004; accepted for publication (in revised form) May 20, 2005; published electronically January 6, 2006.
http://www.siam.org/journals/siopt/16-3/60552.html

†Department of Electrical Engineering, University of California at Berkeley, Berkeley, CA 94720 (polak@eecs.berkeley.edu). This author's research was supported by the National Science Foundation under grant ECS-9900985.

‡United Technologies Research Center, East Hartford, CT 06108 (WetterM@utrc.utc.com). This author's research was performed at the Simulation Research Group, Building Technologies Department, Environmental Energy Technologies Division, Lawrence Berkeley National Laboratory, Berkeley, CA 94720. This author's research was supported by the Assistant Secretary for Energy Efficiency and Renewable Energy, Office of the Building Technologies Program of the U.S. Department of Energy, under contract DE-AC03-76SF00098.

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

In 1997, Torczon [26] was the first to show that all the existing pattern search algorithms are specific implementations of an abstract pattern search scheme and to establish that for unconstrained problems with smooth cost functions $f\colon \mathbb{R}^n \to \mathbb{R}$, the gradient of the cost function vanishes at accumulation points of sequences constructed by this scheme. Lewis and Torczon extended her theory to address bound constrained problems [20] and problems with linear inequality constraints [21]. In both cases, convergence to a feasible point x^* satisfying $\langle \nabla f(x^*), x - x^* \rangle \geq 0$ for all feasible x is proven under the condition that $f(\cdot)$ is once continuously differentiable. Audet and Dennis [3] present a simpler abstraction of GPS algorithms, and, in addition to reestablishing the Torczon and the Lewis and Torczon results, they relax the assumption that the cost function is smooth to that it is locally Lipschitz continuous.

In principle, a natural area for the application of GPS algorithms is engineering optimization, where the cost functions are defined on the solution of complex systems of equations including implicit equations, ordinary differential equations, and partial differential equations. However, in such cases, obtaining an accurate approximation to the cost function often takes many hours, and there is no straightforward way of approximating gradients. Furthermore, it is not uncommon that the termination criteria of the numerical solvers introduce discontinuities in the approximations to the cost function. Hence, standard GPS algorithms can only be used heuristically in this context.

In this paper we present a modified class of GPS algorithms which adjust the precision of the cost function evaluations adaptively: low precision in the early iterations, with precision progressively increasing as a solution is approached. The modified GPS algorithms converge to stationary points of the cost function even though the cost function is approximated by a family of discontinuous functions.

We assume that the cost function $f(\cdot)$ is at least locally Lipschitz continuous and that it can be approximated by a family of functions, say $\{f^*(\epsilon, \cdot)\}_{\epsilon \in \mathbb{R}^q_+}$, with fixed $q \in \mathbb{N}$, where $\epsilon \in \mathbb{R}^q_+$ denotes the tolerance settings of the PDE, ODE, and algebraic equation solvers, and each $f^*(\epsilon, \cdot)$ may be discontinuous but converges to $f(\cdot)$ uniformly on compact sets. A test in the algorithm determines when precision must be increased. This test includes parameters that can be used to control the speed with which precision is increased. We will show by numerical experiments that this flexibility can be exploited to obtain a significant reduction in computation times, as compared to using high precision throughout the computation.

Under the assumption that the cost function is continuously differentiable, all the accumulation points constructed by our GPS algorithms are stationary, while under the assumption that $f(\cdot)$ is only locally Lipschitz continuous, our algorithms converge to points at which the Clarke generalized directional derivatives are nonnegative in predefined directions. Thus, we regain the results of [3].

To prove that our GPS algorithms construct accumulation points at which Clarke's generalized directional derivatives are nonnegative in predefined directions, we made the assumption that $f(\cdot)$ is Lipschitz continuous on $\mathbb{R}^n$ rather than only near the accumulation points, even though local Lipschitz continuity suffices for proving Theorem 5.5. The reason is that in setting up engineering optimization problems, it is easier to construct $f(\cdot)$ to be Lipschitz continuous on $\mathbb{R}^n$, rather than only near the accumulation points, because they are unknown at the time the optimization problem is set up. Furthermore, as shown in [3], strict differentiability, as defined in [8], near an accumulation point x^* suffices to prove that $\nabla f(x^*) = 0$. But since the implicit function theorem and standard theory of differential equations [23] state smoothness

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

of solutions in terms of continuous differentiability rather than strict differentiability, we assume in proving Theorem 5.6 that $f(\cdot)$ is continuously differentiable.

Contrary to the model management framework with generalized pattern search algorithms [10, 27, 25, 4], we do not assume that function values of $f(\cdot)$ are available, and consequently we do not construct surrogate models of increasing accuracy that are based on support points at which $f(\cdot)$ has been evaluated. Our algorithms construct an infinite sequence of approximating cost functions $\{f^*(\epsilon,\cdot)\}_{\epsilon\in\mathbb{R}^q_+}$ so that $f^*(\epsilon,\cdot)$ converges to $f(\cdot)$ fast enough near stationary points. However, since our GPS algorithms include global search and local search stages, as is typical in GPS algorithms, our GPS algorithms allow the use of surrogate models of $f^*(\epsilon,\cdot)$ to obtain points for the global search.

In implicit filtering [16], Kelley accounts for the situation where $f(\cdot)$ is approximated numerically using a computer program that contains adaptive solvers. In implicit filtering, however, one does not adaptively control the error of the cost function evaluations, but rather assumes in proving convergence to a stationary point of $f(\cdot)$ that the error of the approximating cost function decays faster to zero than the step size used in the finite difference approximation of the gradient of the cost function. Because in our convergence analysis we establish a lower bound for Clarke's generalized directional derivative, which we bound by a sequence of finite difference approximations, we need to assume the same rate of error decay as is assumed in implicit filtering. However, in contrast to implicit filtering, our algorithms adaptively control the approximation error. This allows us to construct a simulation precision control algorithm that causes the optimization to use computationally cheap coarse precision approximations to the cost function in the early iterations, and progressively use higher precision cost function evaluations as needed when the algorithm approaches a stationary point.

2. Notation.

1. We denote by $\mathbb{Z}$ the set of integers, by $\mathbb{Q}$ the set of rational numbers, and by $\mathbb{N} \triangleq \{0, 1, \dots\}$ the set of natural numbers. The set $\mathbb{N}_+$ is defined as $\mathbb{N}_+ \triangleq \{1, 2, \dots\}$. Similarly, vectors in $\mathbb{R}^n$ with strictly positive elements are denoted by $\mathbb{R}^n_+ \triangleq \{x \in \mathbb{R}^n \mid x^i > 0, \text{for all } i \in \{1,\dots,n\}\}$ and the set $\mathbb{Q}_+$ is defined as $\mathbb{Q}_+ \triangleq \{q \in \mathbb{Q} \mid q > 0\}$.
2. The inner product in $\mathbb{R}^n$ is denoted by $\langle\cdot,\cdot\rangle$ and for $x, y \in \mathbb{R}^n$ it is defined by $\langle x, y\rangle \triangleq \sum_{i=1}^n x^i\, y^i$.
3. For $\epsilon \in \mathbb{R}^q_+$, by $\epsilon \le \epsilon_{\mathbf{S}}$, we mean that $0 < \epsilon^i \le \epsilon^i_{\mathbf{S}}$ for all $i = \{1,\dots,q\}$.
4. If a subsequence $\{x_i\}_{i\in\mathbf{K}} \subset \{x_i\}_{i=0}^\infty$ converges to some point x, we write $x_i \to^{\mathbf{K}} x$.
5. Let $\mathbb{W}$ be a set containing a sequence $\{w_i\}_{i=0}^k$. Then, we denote by $\underline{w}_k$ the sequence $\{w_i\}_{i=0}^k$ and by $\underline{\mathbf{W}}_k$ the set of all $k+1$ element sequences in $\mathbb{W}$.
6. We denote by $\{e_i\}_{i=1}^n$ the unit vectors in $\mathbb{R}^n$.
7. If $\mathbf{X}$ is a set, we denote by $\partial\mathbf{X}$ its boundary.
8. If $\mathbf{S}$ is a set, we denote by $2^{\mathbf{S}}$ the set of all nonempty subsets of $\mathbf{S}$.
9. If $\mathcal{H}$ is a set, we denote by $\text{card}(\mathcal{H})$ its cardinality.
10. If $D \in \mathbb{Q}^{n\times q}$ is a matrix, we will use the notation $d \in D$ to denote the fact that $d \in \mathbb{Q}^n$ is a column vector of the matrix D.
11. For $s \in \mathbb{R}$, we define $\lfloor s\rfloor \triangleq \max\{k \in \mathbb{Z} \mid k \le s\}$.

3. Minimization problem.

To best explain our precision control algorithm without having to discuss technical details of constructing search directions that con-

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

form, in the sense of [3], to the feasible set of design parameters, and to avoid having to discuss the problem of degenerate search directions, we restrict our discussion to box-constrained problems rather than to problems with general linear constraints. The construction of search directions for linearly constrained problems is discussed in [17, 3, 24].

We will consider box-constrained problems

$$\min_{x\in\mathbf{X}} f(x), \tag{1a}$$

$$\mathbf{X} \triangleq \left\{x \in \mathbb{R}^n \mid l^i \le x^i \le u^i,\ i \in \{1,\dots,n\}\right\}, \tag{1b}$$

with $-\infty \le l^i < u^i \le \infty$ for $i \in \{1,\dots,n\}$, where the cost function $f\colon \mathbb{R}^n \to \mathbb{R}$ is (at least) Lipschitz continuous.

We assume that the function $f(\cdot)$ cannot be evaluated exactly, but that it can be approximated by functions $f^*\colon \mathbb{R}^q_+ \times \mathbb{R}^n \to \mathbb{R}$, and that $\epsilon \in \mathbb{R}^q_+$ is a vector of fixed dimension $q \in \mathbb{N}$ that contains the tolerance settings of the PDE, ODE, and algebraic equation solvers. We will assume that $f(\cdot)$ and its approximating functions $\{f^*(\epsilon,\cdot)\}_{\epsilon\in\mathbb{R}^q_+}$ have the following properties.

Assumption 3.1.

1. There exists an error bound function $\varphi\colon \mathbb{R}^q_+ \to \mathbb{R}_+$ such that for any compact set $\mathbf{S} \subset \mathbf{X}$, there exists an $\epsilon_{\mathbf{S}} \in \mathbb{R}^q_+$ and a scalar $K_{\mathbf{S}} \in (0,\infty)$ such that for all $x \in \mathbf{S}$ and for all $\epsilon \in \mathbb{R}^q_+$, with $\epsilon \le \epsilon_{\mathbf{S}}$,

$$|\, f^*(\epsilon,x) - f(x)| \le K_{\mathbf{S}}\, \varphi(\epsilon). \tag{2a}$$

Furthermore,

$$\lim_{\|\epsilon\|\to 0} \varphi(\epsilon) = 0. \tag{2b}$$

2. The function $f\colon \mathbb{R}^n \to \mathbb{R}$ is at least locally Lipschitz continuous on $\mathbf{X}$.

Remark 3.2. The functions $\{f^*(\epsilon,\cdot)\}_{\epsilon\in\mathbb{R}^q_+}$ may be discontinuous.

Examples of error bound functions $\varphi(\cdot)$ can be found in section 6 and in [24].

Next, we state an assumption on the level sets of the family of approximating cost functions. To do so, we first define the notion of a level set.

Definition 3.3 (level set). *Given a function $f\colon \mathbb{R}^n \to \mathbb{R}$ and an $\alpha \in \mathbb{R}$, we will say that the set $\mathbf{L}_\alpha(f) \subset \mathbb{R}^n$, defined as*

$$\mathbf{L}_\alpha(f) \triangleq \{x \in \mathbb{R}^n \mid f(x) \le \alpha\}, \tag{3}$$

is a level set *of $f(\cdot)$, parametrized by α.*

Assumption 3.4 (compactness of level sets). Let $\{f^*(\epsilon,\cdot)\}_{\epsilon\in\mathbb{R}^q_+}$ be as in Assumption 3.1 and let $\mathbf{X} \subset \mathbb{R}^n$ be the constraint set. Let $x_0 \in \mathbf{X}$ be the initial iterate and $\epsilon_0 \in \mathbb{R}^q_+$ be the initial solver tolerance. Then, we assume that there exists a compact set $\mathbf{C} \subset \mathbb{R}^n$ such that for all $\epsilon \in \mathbb{R}^q_+$, with $\epsilon \le \epsilon_0$,

$$\mathbf{L}_{f^*(\epsilon_0,x_0)}\big(f^*(\epsilon,\cdot)\big) \cap \mathbf{X} \subset \mathbf{C}. \tag{4}$$

4. Precision control for generalized pattern search algorithms.

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

4.1. Characterization of generalized pattern search algorithms. There exist different geometrical characterizations for pattern search algorithms, and a general framework is presented in the review [17]. To focus on the explanation of our precision control algorithms without having to repeat the excellent discussions in [17], we will use a simple implementation of pattern search algorithms to explain our precision control algorithms. In particular, we will assume that the search directions are the columns of the matrix

$$D \triangleq [-e_1,\, +e_1,\, \ldots\,, -e_n,\, +e_n] \in \mathbb{Z}^{n\times 2n}, \tag{5}$$

which suffices for box-constrained problems. Furthermore, we assume that the sequence of mesh size parameters, which parameterize the minimum distance between iterates, is constructed as follows.

Assumption 4.1 (kth mesh size parameter). Let $r, s_0, k \in \mathbb{N}$, with $r > 1$, and $\{t_i\}_{i=0}^{k-1} \subset \mathbb{N}$. We assume that the sequence of mesh size parameters $\{\Delta_k\}_{k=0}^{\infty}$ satisfies

$$\Delta_k \triangleq \frac{1}{r^{s_k}}, \tag{6a}$$

where for $k > 0$

$$s_k \triangleq s_0 + \sum_{i=0}^{k-1} t_i. \tag{6b}$$

With this construction, all iterates lie on nested rational meshes of the form

$$\mathbb{M}_k \triangleq \{x_0 + \Delta_k\, D\, m \mid m \in \mathbb{N}^{2n}\}. \tag{7}$$

We will now characterize the set-valued maps that determine the mesh points for the "global" and "local" searches.

Definition 4.2. *Let $\underline{\mathbf{X}}_k \subset \mathbb{R}^n$ and $\underline{\mathbf{\Delta}}_k \subset \mathbb{Q}_+$ be the sets of all sequences containing $k+1$ elements, let $\mathbb{M}_k$ be the current mesh as defined in* (7)*, and let $\epsilon_k \in \mathbb{R}^q_+$ be the solver tolerance.*

1. *We define the* global search set map *to be any set-valued map*

$$\gamma_k \colon \underline{\mathbf{X}}_k \times \underline{\mathbf{\Delta}}_k \times \mathbb{R}^q_+ \to \left(2^{\mathbb{M}_k} \cap \mathbf{X}\right) \cup \emptyset \tag{8a}$$

   whose image $\gamma_k(\underline{x}_k, \underline{\Delta}_k, \epsilon_k)$ contains only a finite number of mesh points.
2. *We will call $\mathcal{G}_k \triangleq \gamma_k(\underline{x}_k, \underline{\Delta}_k, \epsilon_k)$ the* global search set.
3. *We define the directions for the local search as $D \triangleq [-e_1,\, +e_1,\, \ldots\,, -e_n,\, +e_n]$.*
4. *We will call*

$$\mathcal{L}_k \triangleq \left\{x_k \pm \Delta_k\, e_i \mid i \in \{1, \ldots, n\}\right\} \cap \mathbf{X} \tag{8b}$$

   the local search set.

Remark 4.3.

1. As we shall see, the global search affects only the efficiency of the algorithm but not its convergence properties. Any heuristic procedure that leads to a finite number of function evaluations can be used for $\gamma_k(\cdot,\cdot,\cdot)$. Thus, the elements in $\mathcal{G}_k$ can be determined using a search procedure on surrogate cost functions, as in [10, 27, 25, 4], if the search procedure is a finite process.
2. The empty set is included in the range of $\gamma_k(\cdot,\cdot,\cdot)$ to allow omitting the global search.

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

4.2. Adaptive precision GPS algorithm models. We will now present our GPS algorithm models with adaptive precision cost function evaluations. We will first present an algorithm that simultaneously decreases Δ_k and ϵ_k.

ALGORITHM 4.4 (GPS algorithm model with simultaneous decrease of ϵ_k and Δ_k).

Data: Sufficient decrease parameter $\zeta \geq 0$;
Initial iterate $x_0 \in \mathbf{X}$;
Mesh size divider $r \in \mathbb{N}$, with $r > 1$;
Initial mesh size exponent $s_0 \in \mathbb{N}$.

Maps: Global search set map $\gamma_k \colon \underline{\mathbf{X}_k} \times \underline{\mathbf{\Delta}}_k \times \mathbb{R}^q_+ \to \left(2^{\mathbb{M}_k} \cap \mathbf{X}\right) \cup \emptyset$;
$\varphi \colon \mathbb{R}^q_+ \to \mathbb{R}_+$ as in Assumption 3.1;
Function $\rho \colon \mathbb{R}_+ \to \mathbb{R}^q_+$ (to assign ϵ_k), such that the composition $\varphi \circ \rho \colon \mathbb{R}_+ \to \mathbb{R}_+$ is strictly monotone increasing and satisfies $\varphi(\rho(\Delta))/\Delta \to 0$, as $\Delta \to 0$.

Step 0: Initialize $k = 0$, $\Delta_0 = 1/r^{s_0}$, and $\epsilon_0 = \rho(1)$.

Step 1: Global Search
Construct the global search set $\mathcal{G}_k = \gamma_k(\underline{x}_k, \underline{\Delta}_k, \epsilon_k)$.
If $f^*(\epsilon_k, x') - f^*(\epsilon_k, x_k) < -\zeta\, \varphi(\epsilon_k)$ for any $x' \in \mathcal{G}_k$, go to Step 3;
else, go to Step 2.

Step 2: Local Search
Evaluate $f^*(\epsilon_k, \cdot)$ for any $x' \in \mathcal{L}_k$ until some $x' \in \mathcal{L}_k$ satisfying $f^*(\epsilon_k, x') - f^*(\epsilon_k, x_k) < -\zeta\, \varphi(\epsilon_k)$ is obtained, or until all points in $\mathcal{L}_k$ are evaluated.

Step 3: Parameter Update
If there exists an $x' \in \mathcal{G}_k \cup \mathcal{L}_k$ satisfying $f^*(\epsilon_k, x') - f^*(\epsilon_k, x_k) < -\zeta\, \varphi(\epsilon_k)$, set $x_{k+1} = x'$, $s_{k+1} = s_k$, $\Delta_{k+1} = \Delta_k$, and $\epsilon_{k+1} = \epsilon_k$;
else, set $x_{k+1} = x_k$, $s_{k+1} = s_k + t_k$, with $t_k \in \mathbb{N}_+$ arbitrary, $\Delta_{k+1} = 1/r^{s_{k+1}}$, and $\epsilon_{k+1} = \rho(\Delta_{k+1}/\Delta_0)$.

Step 4: Replace k by $k+1$, and go to Step 1.

Remark 4.5.

1. We allow setting $\zeta = 0$ to obtain a GPS algorithm without imposing a sufficient decrease condition. In proving that $\lim_{k\to\infty} \Delta_k = 0$, we will make use of the fact that the iterates x_k are contained in a compact set and lie on a rational lattice in which the spacing of the elements depends on Δ_k, and hence simple decrease suffices to accept an iterate [17].
2. To ensure that ϵ_0 does not depend on the scaling of Δ_0, we normalize the argument of $\rho(\cdot)$.
3. In Step 2, once a (sufficient) decrease in cost is obtained, one can proceed to Step 3. But note that one is allowed to evaluate the approximating cost function at more points in $\mathcal{L}_k$ in an attempt to obtain a larger decrease in cost. However, one is allowed to proceed to Step 3 only after either a (sufficient) decrease in cost has been obtained or after *all* points in $\mathcal{L}_k$ were tested.
4. In Step 3, one is not restricted to accept the point $x' \in \mathcal{G}_k \cup \mathcal{L}_k$ that gives lowest cost. But the mesh size parameter Δ_k is reduced *only* if there exists no $x' \in \mathcal{G}_k \cup \mathcal{L}_k$ satisfying $f^*(\epsilon_k, x') - f^*(\epsilon_k, x_k) < -\zeta\, \varphi(\epsilon_k)$.
5. To simplify the explanation of our precision control algorithms, we do not increase the mesh size parameter if the cost has been reduced. However, our global search allows searching on a coarser mesh $\widehat{\mathbb{M}} \subset \mathbb{M}_k$, and hence our algorithm can easily be extended to include a rule for increasing Δ_k for a

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

finite number of iterations (see [24]).

6. Audet and Dennis [3] update the mesh size parameter using the formula $\Delta_{k+1} = \tau^{w_k}\,\Delta_k$, where $\tau > 1$ is a rational number that remains constant over all iterations, and where $w_k \in [w^-, w^+] \subset \mathbb{Z}$ for some constants $w^-, w^+ \in \mathbb{Z}$, with $w^- \leq -1$ and $0 \leq w^+$. In the notation of Audet and Dennis, our algorithm sets $\tau = r \in \mathbb{N}_+$, $r > 1$ and $w_k \leq 0$, $w_k \in \mathbb{Z}$. Our GPS algorithms do not require $t_k \in \mathbb{N}_+$ to be bounded from above (i.e., in the notation of Audet and Dennis [3], w_k need not be bounded from below by w^-). However, it is obvious that from a computational point of view, a small t_k should be selected to avoid too fine a mesh in early iterations. We prefer presenting our GPS algorithms with our construction of mesh size dividers because it leads to simpler convergence proofs that better highlight our precision control scheme.

In [24], we use the GPS Algorithm model 4.4 with $\zeta = 0$ to extend the Hooke–Jeeves algorithm for use with adaptive precision cost function evaluations.

We will now present a modification of the GPS Algorithm model 4.4 in which we decrease Δ_k only after $\varphi(\epsilon_k)$ has been sufficiently decreased.

Algorithm 4.6 (GPS algorithm model).

Data: Parameters $\alpha \in (0, 1)$ and $\zeta \geq 0$;
Initial iterate $x_0 \in \mathbf{X}$;
Mesh size divider $r \in \mathbb{N}$, with $r > 1$;
Initial mesh size exponent $s_0 \in \mathbb{N}$.

Maps: Global search set map $\gamma_k \colon \underline{\mathbf{X}_k} \times \underline{\mathbf{\Delta}_k} \times \mathbb{R}^q_+ \to \left(2^{\mathbb{M}_k} \cap \mathbf{X}\right) \cup \emptyset$;
$\varphi \colon \mathbb{R}^q_+ \to \mathbb{R}_+$ as in Assumption 3.1;
$\rho \colon \mathbb{N} \to \mathbb{R}^q_+$ (to assign ϵ_k) such that the composition $\varphi \circ \rho \colon \mathbb{N} \to \mathbb{R}_+$ is strictly monotone decreasing and satisfies $\varphi(\rho(N)) \to 0$, as $N \to \infty$.

Step 0: Initialize $k = 0$, $\Delta_0 = 1/r^{s_0}$, $N = 1$, and $\epsilon_0 = \rho(1)$.

Step 1: Global Search
Construct the global search set $\mathcal{G}_k = \gamma_k(\underline{x}_k, \underline{\Delta}_k, \epsilon_k)$.
If $f^*(\epsilon_k, x') - f^*(\epsilon_k, x_k) < -\zeta\,\varphi(\epsilon_k)$ for any $x' \in \mathcal{G}_k$, go to Step 3;
else, go to Step 2.

Step 2: Local Search
Evaluate $f^*(\epsilon_k, \cdot)$ for any $x' \in \mathcal{L}_k$ until some $x' \in \mathcal{L}_k$ satisfying $f^*(\epsilon_k, x') - f^*(\epsilon_k, x_k) < -\zeta\,\varphi(\epsilon_k)$ is obtained, or until all points in $\mathcal{L}_k$ are evaluated.

Step 3: Parameter Update
If there exists an $x' \in \mathcal{G}_k \cup \mathcal{L}_k$ satisfying $f^*(\epsilon_k, x') - f^*(\epsilon_k, x_k) < -\zeta\,\varphi(\epsilon_k)$,
    set $x_{k+1} = x'$, $s_{k+1} = s_k$, $\Delta_{k+1} = \Delta_k$, and $\epsilon_{k+1} = \epsilon_k$, do not change N, and go to Step 4;
else,
    replace N by $N + 1$ and set $\epsilon_{k+1} = \rho(N)$.
    If $\varphi(\epsilon_{k+1})^\alpha/\Delta_k < \Delta_k$,
        set $s_{k+1} = s_k + t_k$, with $t_k \in \mathbb{N}_+$ large enough such that $\varphi(\epsilon_{k+1})^\alpha/\Delta_{k+1} \geq \Delta_{k+1}$ (with $\Delta_{k+1} = 1/r^{s_{k+1}}$), and set $\Delta_{k+1} = 1/r^{s_{k+1}}$;
    else,

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

set $s_{k+1} = s_k$ and $\Delta_{k+1} = \Delta_k$.
Set $x_{k+1} = x_k$, and go to Step 4.

Step 4: Replace k by $k+1$, and go to Step 1.

Remark 4.7.

1. In Step 3, the mesh refinement is well defined because there always exists a $t_k \in \mathbb{N}_+$ for which $\varphi(\epsilon_{k+1})^\alpha \geq \Delta_{k+1}^2$, namely any $t_k \in \mathbb{N}_+$ that satisfies

$$t_k \geq \frac{2 \log \Delta_k - \alpha \log \varphi(\epsilon_{k+1})}{2 \log r}. \tag{9}$$

2. In Step 3, the test $\varphi(\epsilon_{k+1})^\alpha / \Delta_k < \Delta_k$, with $\alpha \in (0, 1)$, ensures that $\varphi(\epsilon_k)/\Delta_k \to 0$, as $\Delta_k \to 0$, which is essential for proving convergence.
3. The algorithm parameter α can be used to control how fast Δ_k is decreased. The smaller $\alpha \in (0, 1)$, the later Δ_k is decreased.

5. Convergence analysis.

5.1. Unconstrained minimization. We will now establish the convergence properties of the GPS Algorithm models 4.4 and 4.6 on unconstrained minimization problems, i.e., for $\mathbf{X} = \mathbb{R}^n$.

The following obvious result will be used to show that $\Delta_k \to 0$ as $k \to \infty$.

PROPOSITION 5.1. *Any bounded subset of a mesh $\mathbb{M}_k$ contains only a finite number of mesh points.*

PROPOSITION 5.2. *Suppose that Assumption* 3.4 *is satisfied and let $\{\Delta_k\}_{k=0}^\infty \subset \mathbb{Q}_+$ be the sequence of mesh size parameters constructed by GPS Algorithm model* 4.4 *or* 4.6. *Then,* $\lim_{k\to\infty} \Delta_k = 0$.

Proof. We first prove the proposition for the GPS Algorithm model 4.4. By (6a), $\Delta_k = 1/r^{s_k}$, where $r \in \mathbb{N}$ with $r > 1$ and $\underline{s}_k \subset \mathbb{N}$ is a nondecreasing sequence. For the sake of contradiction, suppose that there exists a $\Delta_{k^*} \in \mathbb{Q}_+$ such that $\Delta_k \geq \Delta_{k^*}$ for all $k \in \mathbb{N}$. Then there exists a corresponding $s_{k^*} = \max_{k\in\mathbb{N}} s_k$, and the finest possible mesh is $\mathbb{M}_{k^*} \triangleq \{x_0 + (1/r^{s_{k^*}})\, D\, m \,|\, m \in \mathbb{N}^{2n}\}$.

Next, since by Assumption 3.4 there exists a compact set $\mathbf{C}$, such that $\mathbf{L}_{f^*(\epsilon_0,x_0)}\big(f^*(\epsilon,\cdot)\big) \subset \mathbf{C}$ for all $\epsilon \in \mathbb{R}_+^q$, with $\epsilon \leq \epsilon_0 = \rho(1)$, it follows from Proposition 5.1 that $\mathbb{M}_{k^*} \cap \mathbf{L}_{f^*(\epsilon_0,x_0)}\big(f^*(\epsilon_k,\cdot)\big)$ contains only a finite number of mesh points for any $\epsilon_k \in \mathbb{R}_+^q$, with $\epsilon_k \leq \rho(1)$. Thus, at least one point in $\mathbb{M}_{k^*}$ must belong to the sequence $\{x_k\}_{k=0}^\infty$ infinitely many times. Furthermore, because $\{s_k\}_{k=0}^\infty \subset \mathbb{N}$ is nondecreasing with s_{k^*} being its maximal element, it follows that $\epsilon_k = \epsilon_{k^*} = \rho(\Delta_{k^*}/\Delta_0)$ for all iterations $k \geq k^*$. Hence the sequence $\{f^*(\epsilon_k, x_k)\}_{k=k^*}^\infty$ cannot satisfy $f^*(\epsilon_k, x_{k+1}) - f^*(\epsilon_k, x_k) < -\zeta\, \varphi(\epsilon_k)$ for all $k \geq k^*$, which contradicts the constructions in Algorithm 4.4.

We now prove the proposition for the GPS Algorithm model 4.6. Suppose $\lim_{k\to\infty} \Delta_k \neq 0$. Then, there exists only a finite number of iterations in which there exists no $x' \in \mathcal{G}_k \cup \mathcal{L}_k$ that satisfies $f^*(\epsilon_k, x') - f^*(\epsilon_k, x_k) < -\zeta\, \varphi(\epsilon_k)$, because otherwise N is replaced by $N+1$ an infinite number of times in Step 3, from which follows that $\varphi(\rho(N))^\alpha \to 0$, as $N \to \infty$, and hence $\Delta_k \to 0$, as $k \to \infty$. Thus, there exists an $N^* \in \mathbb{N}$ and a corresponding $k^* \in \mathbb{N}$ such that $N \leq N^*$, $\Delta_k = \Delta_{k^*}$, and $\epsilon_k = \epsilon_{k^*} = \rho(N^*)$ for all $k \geq k^*$, and the finest possible mesh is $\mathbb{M}_{k^*} \triangleq \big\{x_0 + \Delta_{k^*}\, D\, m \mid m \in \mathbb{N}^{2\,n}\big\}$.

By Assumption 3.4, there exists a compact set $\mathbf{C}$, such that $\mathbf{L}_{f^*(\epsilon_0,x_0)}\big(f^*(\epsilon,\cdot)\big) \subset \mathbf{C}$, for all $\epsilon \in \mathbb{R}_+^q$, with $\epsilon \leq \epsilon_0 = \rho(1)$. Hence, it follows from Proposition 5.1 that

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

$\mathbb{M}_{k^*} \cap \mathbf{L}_{f^*(\epsilon_0,x_0)}\big(f^*(\epsilon_k,\cdot)\big)$ contains only a finite number of mesh points for all $\epsilon_k \le \epsilon_0$. Thus, at least one point in $\mathbb{M}_{k^*}$ must belong to the sequence $\{x_k\}_{k=0}^{\infty}$ infinitely many times. Hence, the sequence $\{f^*(\epsilon_k,x_k)\}_{k=k^*}^{\infty}$ cannot satisfy $f^*(\epsilon_k,x_{k+1}) - f^*(\epsilon_k,x_k) < -\zeta\,\varphi(\epsilon_k)$ for all $k \ge k^*$, which contradicts the constructions in Algorithm 4.6. ∎

Having shown that $\lim_{k\to\infty}\Delta_k = 0$, we can use the notion of a refining subsequence as introduced by Audet and Dennis [3].

DEFINITION 5.3 (refining subsequence). *Consider a sequence $\{x_k\}_{k=0}^{\infty}$ constructed by GPS Algorithm model* 4.4 *or* 4.6. *We will say that the subsequence $\{x_k\}_{k\in\mathbf{K}}$ is the* refining subsequence, *if $\Delta_{k+1} < \Delta_k$ for all $k \in \mathbf{K}$, and $\Delta_{k+1} = \Delta_k$ for all $k \notin \mathbf{K}$.*

When the cost function $f(\cdot)$ is only locally Lipschitz continuous, we, as well as Audet and Dennis [3], only get a weak characterization of limit points of the refining subsequence, as we will now see.

We recall the definition of Clarke's generalized directional derivative [8].

DEFINITION 5.4 (Clarke's generalized directional derivative). *Let $f\colon \mathbb{R}^n \to \mathbb{R}$ be locally Lipschitz continuous at the point $x^* \in \mathbb{R}^n$. Then,* Clarke's generalized directional derivative *of $f(\cdot)$ at x^* in the direction $h \in \mathbb{R}^n$ is defined by*

$$f^\circ(x^*;\,h) \triangleq \limsup_{\substack{x\to x^*\\ t\downarrow 0}} \frac{f(x+t\,h)-f(x)}{t}. \tag{10}$$

THEOREM 5.5. *Suppose that Assumptions* 3.1 *and* 3.4 *are satisfied, let D be as in Definition* 4.2, *and let $x^* \in \mathbb{R}^n$ be an accumulation point of the refining subsequence $\{x_k\}_{k\in\mathbf{K}}$ constructed by GPS Algorithm model* 4.4 *or* 4.6. *Then, for all $d \in D$,*

$$f^\circ(x^*;\,d) \ge 0. \tag{11}$$

Proof. The proof is identical for both algorithms. Let $\{x_k\}_{k\in\mathbf{K}}$ be the refining subsequence and, without loss of generality, suppose that $x_k \to^{\mathbf{K}} x^*$. By Assumption 3.4, there exists a compact set $\mathbf{C}$ such that $\mathbf{L}_{f^*(\epsilon_0,x_0)}\big(f^*(\epsilon,\cdot)\big) \subset \mathbf{C}$ for all $\epsilon \in \mathbb{R}^q_+$, with $\epsilon \le \epsilon_0 = \rho(1)$. Therefore, by Assumption 3.1, there exists an $\epsilon_{\mathbf{L}} \in \mathbb{R}^q_+$ and a scalar $K_{\mathbf{L}} \in (0,\,\infty)$ such that, for all $x \in \mathbf{C}$ and for all $\epsilon \in \mathbb{R}^q_+$, with $\epsilon \le \epsilon_{\mathbf{L}}$, we have $|f^*(\epsilon,x) - f(x)| \le K_{\mathbf{L}}\,\varphi(\epsilon)$. Because $f(\cdot)$ is locally Lipschitz continuous, its directional derivative $f^\circ(\cdot;\cdot)$ exists. The precision control schemes of Algorithms 4.4 and 4.6 both imply that $\epsilon_k \to^{\mathbf{K}} 0$, and furthermore that $f^*(\epsilon_k, x_k + \Delta_k\,d) - f^*(\epsilon_k,x_k) \ge -\zeta\,\varphi(\epsilon_k)$ for all $d \in D$ and for all $k \in \mathbf{K}$. Hence, for any $d \in D$,

$$\begin{aligned}
f^\circ(x^*;\,d) &\triangleq \limsup_{\substack{x\to x^*\\ t\downarrow 0}} \frac{f(x+t\,d)-f(x)}{t}\\
&\ge \limsup_{k\in\mathbf{K}} \frac{f(x_k+\Delta_k\,d)-f(x_k)}{\Delta_k}\\
&\ge \limsup_{k\in\mathbf{K}} \frac{f^*(\epsilon_k,x_k+\Delta_k\,d)-f^*(\epsilon_k,x_k)-2\,K_{\mathbf{L}}\,\varphi(\epsilon_k)}{\Delta_k}\\
&\ge \limsup_{k\in\mathbf{K}} \frac{f^*(\epsilon_k,x_k+\Delta_k\,d)-f^*(\epsilon_k,x_k)}{\Delta_k} - \limsup_{k\in\mathbf{K}} 2\,K_{\mathbf{L}}\,\frac{\varphi(\epsilon_k)}{\Delta_k}\\
&\ge -\limsup_{k\in\mathbf{K}} \zeta\,\frac{\varphi(\epsilon_k)}{\Delta_k} - \limsup_{k\in\mathbf{K}} 2\,K_{\mathbf{L}}\,\frac{\varphi(\epsilon_k)}{\Delta_k}.
\end{aligned} \tag{12}$$

The second line follows from the first because the choice of x and t is restricted to $\{x_k\}_{k\in\mathbf{K}}$ and $\{\Delta_k\}_{k\in\mathbf{K}}$. Since by Proposition 5.2 $\Delta_k \to^{\mathbf{K}} 0$, it follows from the constructions in GPS Algorithm models 4.4 and 4.6 that $\varphi(\epsilon_k)/\Delta_k \to^{\mathbf{K}} 0$. ∎

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

We now state that pattern search algorithms with adaptive precision cost function evaluations converge to stationary points.

THEOREM 5.6 (convergence to a stationary point). *Suppose that Assumptions* 3.1 *and* 3.4 *are satisfied and, in addition, that* $f(\cdot)$ *is once continuously differentiable. Let* $x^* \in \mathbb{R}^n$ *be an accumulation point of the refining subsequence* $\{x_k\}_{k\in\mathbf{K}}$*, constructed by GPS Algorithm model* 4.4 *or* 4.6*. Then,*

$$\nabla f(x^*) = 0. \tag{13}$$

Proof. Because $f(\cdot)$ is once continuously differentiable, we have $f^\circ(x^*;\, h) = df(x^*; h) = \langle \nabla f(x^*),\, h\rangle$ for all $h \in \mathbb{R}^n$. It follows from Theorem 5.5 that $0 \leq \langle \nabla f(x^*),\, d\rangle$ for all $d \in D$, with D as in Definition 4.2. We can express any $h \in \mathbb{R}^n$ as

$$h = \sum_{i=1}^{2\,n} \alpha_i\, d_i, \quad d_i \in D, \quad \alpha_i \geq 0 \quad \forall\, i \in \{1, \ldots, 2\,n\}. \tag{14a}$$

Hence, $0 \leq \langle \nabla f(x^*),\, h\rangle$. Similarly, we can express the vector $-h$, as follows:

$$-h = \sum_{i=1}^{2\,n} \beta_i\, d_i, \quad d_i \in D, \quad \beta_i \geq 0 \quad \forall\, i \in \{1, \ldots, 2\,n\}. \tag{14b}$$

Hence, $0 \geq \langle \nabla f(x^*),\, h\rangle$, which implies that $0 = \langle \nabla f(x^*),\, h\rangle$ and, since h is arbitrary, that $\nabla f(x^*) = 0$. ∎

5.2. Constrained minimization. We will now extend our convergence proofs to the box-constrained problem (1). First, we introduce the notion of a tangent cone and a normal cone, which are defined as follows.

DEFINITION 5.7 (tangent and normal cone).

1. *Let* $\mathbf{X} \subset \mathbb{R}^n$ *be defined as in* (1b)*. Then, we define the* tangent cone *to* $\mathbf{X}$ *at a point* $x^* \in \mathbf{X}$ *by*

$$\mathbf{T}_{\mathbf{X}}(x^*) \triangleq \{\mu\,(x - x^*) \mid \mu \geq 0,\, x \in \mathbf{X}\}. \tag{15a}$$

2. *Let* $\mathbf{T}_{\mathbf{X}}(x^*)$ *be as above. Then, we define the* normal cone *to* $\mathbf{X}$ *at* $x^* \in \mathbf{X}$ *by*

$$\mathbf{N}_{\mathbf{X}}(x^*) \triangleq \{v \in \mathbb{R}^n \mid \forall\, t \in \mathbf{T}_{\mathbf{X}}(x^*),\, \langle v,\, t\rangle \leq 0\}. \tag{15b}$$

Next, for $x^* \in \mathbf{X}$ we define the subset of column vectors of the search direction matrix $D \triangleq [-e_1,\, +e_1,\, \ldots,\, -e_n,\, +e_n]$ that is required to generate the tangent cone $\mathbf{T}_{\mathbf{X}}(x^*)$. This will facilitate the extension of Theorem 5.5 to box-constrained problems.

DEFINITION 5.8. *Let* $\mathbf{X} \subset \mathbb{R}^n$ *be as in* (1b) *and let* $\mathbf{T}_{\mathbf{X}}(\cdot)$ *be as in Definition* 5.7*. For* $x^* \in \mathbf{X}$*, we define* $\mathcal{H}(x^*) \subset \{-e_1, +e_1, \ldots, -e_n, +e_n\}$ *such that* $\mathbf{T}_{\mathbf{X}}(x^*) = \{\sum_{i=1}^{\operatorname{card}\mathcal{H}(x^*)} \alpha^i\, h_i \mid h_i \in \mathcal{H}(x^*),\, \alpha^i \geq 0,\, i \in \{1, \ldots, \operatorname{card}\mathcal{H}(x^*)\}\}$.

THEOREM 5.9. *Suppose that Assumptions* 3.1 *and* 3.4 *are satisfied. Let* $x^* \in \mathbf{X}$ *be an accumulation point of the refining subsequence* $\{x_k\}_{k\in\mathbf{K}}$ *constructed by GPS Algorithm model* 4.4 *or* 4.6 *in solving problem* (1) *and let* $\mathcal{H}(x^*)$ *be as defined in Definition* 5.8*. Then,*

$$f^\circ(x^*;\, d) \geq 0 \qquad \forall\, d \in \mathcal{H}(x^*). \tag{16}$$

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Proof. If x^* is in the interior of $\mathbf{X}$, then the result reduces to Theorem 5.5.

Hence, suppose that $x^* \in \partial\mathbf{X}$ and let $\mathcal{H}(x^*)$ be as in Definition 5.8. Because x^* is an accumulation point of $\{x_k\}_{k\in\mathbf{K}}$, there exists an infinite subset $\mathbf{K}' \subset \mathbf{K}$, such that $x_k \to^{\mathbf{K}'} x^*$, and $\{x_k + \Delta_k\, d\}_{k\in\mathbf{K}',\, d\in\mathcal{H}(x^*)} \subset \mathbf{X}$. The precision control schemes of Algorithms 4.4 and 4.6 both imply that $\epsilon_k \to^{\mathbf{K}'} 0$, and furthermore that $f^*(\epsilon_k, x_k + \Delta_k\, d) - f^*(\epsilon_k, x_k) \geq -\zeta\, \varphi(\epsilon_k)$ for all $d \in \mathcal{H}(x^*)$ and for all $k \in \mathbf{K}'$. Thus, for any $d \in \mathcal{H}(x^*)$, (12) still holds, from which we conclude that (16) holds. ∎

We can now state that the GPS Algorithm models 4.4 and 4.6 generate sequences of iterates which contain accumulation points that are feasible stationary points of problem (1).

THEOREM 5.10 (convergence to a feasible stationary point). *Suppose that Assumptions* 3.1 *and* 3.4 *are satisfied and, in addition, that $f(\cdot)$ is once continuously differentiable. Let $x^* \in \mathbf{X}$ be an accumulation point of the refining subsequence $\{x_k\}_{k\in\mathbf{K}}$ constructed by GPS Algorithm model* 4.4 *or* 4.6 *in solving problem* (1) *and let $\mathcal{H}(x^*)$ be as defined in Definition* 5.8. *Then,*

$$\langle \nabla f(x^*),\, t\rangle \geq 0 \qquad \forall\, t \in \mathbf{T}_{\mathbf{X}}(x^*), \tag{17a}$$

and

$$-\nabla f(x^*) \in \mathbf{N}_{\mathbf{X}}(x^*). \tag{17b}$$

Proof. Because $f(\cdot)$ is once continuously differentiable, it follows that $f^\circ(x^*; h) = \langle \nabla f(x^*), h\rangle$ for all $h \in \mathbb{R}^n$. Because $\langle \nabla f(x^*), \cdot\rangle$ is linear and because every $t \in \mathbf{T}_{\mathbf{X}}(x^*)$ can be expressed as a nonnegative linear combination of elements in $\mathcal{H}(x^*)$, we conclude in view of (16) that (17a) and (17b) hold. ∎

6. Numerical experiments. In all numerical experiments, in the GPS Algorithm models 4.4 and 4.6 we set the mesh size divider $r = 2$ and the initial mesh size exponent $s_0 = 0$. If no (sufficient) decrease in cost has been obtained, then we divide the mesh size parameter Δ_k by a factor of two. Hence, if $\mathbf{K}$ denotes the set that contains the iteration indices of the refining subsequence, as defined in Definition 5.3, then in GPS Algorithm models 4.4 and 4.6 $t_k = 1$ for $k \in \mathbf{K}$ and $t_k = 0$ for $k \notin \mathbf{K}$.

All optimization runs were carried out using a 2.2 GHz AMD processor running Linux with the $2.4.18 - 3$ kernel.

6.1. Optimization problem with cost function defined on the solutions of a DAE system. In this numerical experiment, we minimize the annual energy consumption for lighting, cooling, and heating of an office building in Houston, TX. We simulated three characteristic rooms that are representative of the energy consumption of a large office building. The components of the design parameter $x \in \mathbb{R}^n$ are the window sizes for the south and north facing windows, the depth of an overhang placed above the south facing window, and two control setpoints that activate shading devices outside the north and south facing windows, hence $n = 5$.

6.1.1. Exact cost function. The cost function is once continuously differentiable and defined as

$$f(x) \triangleq F\big(z(x, 1)\big), \tag{18}$$

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

where $z(x,1)$ is the solution of a semiexplicit nonlinear DAE system with index one [5] of the form

$$\frac{dz(x,t)}{dt} = h\big(x, z(x,t), \mu\big), \qquad t \in [0,\ 1], \tag{19a}$$

$$z(x,0) = z_0(x), \tag{19b}$$

$$\gamma\big(x, z(x,t), \mu\big) = 0, \tag{19c}$$

where $h\colon \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R}^l \to \mathbb{R}^m$, $z_0\colon \mathbb{R}^n \to \mathbb{R}^m$, and $\gamma\colon \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R}^l \to \mathbb{R}^l$ are once Lipschitz continuously differentiable in all arguments. For all $x \in \mathbb{R}^n$ and for all $z(\cdot,\cdot) \in \mathbb{R}^m$, (19c) has a unique solution $\mu^*(x,z) \in \mathbb{R}^l$ and the matrix with partial derivatives $\partial\gamma(x, z(x,t), \mu^*(x,z))/\partial\mu \in \mathbb{R}^{l\times l}$ is nonsingular. Thus, by using the implicit function theorem and standard theory of ODEs [23] one can show that there exists a unique once continuously differentiable function $z(\cdot,1)$ and hence $f(\cdot)$ is once continuously differentiable. In this experiment, $m = 104$ and $l = 4$, and we defined the function $F(\cdot)$ in (18) as

$$F\big(z(x,1)\big) \triangleq \frac{z^1(x,1)}{\eta_h} + \frac{z^2(x,1)}{\eta_c} + 3\, z^3(x,1), \tag{20}$$

where $z^1(x,1)$ and $z^2(x,1)$ are the annual heating and cooling loads of the rooms, $z^3(x,1)$ is the electricity consumption for lighting the rooms, and $\eta_h = 0.44$ and $\eta_c = 0.77$ are plant efficiencies that relate the annual room load to the primary energy consumption for heating and cooling generation, including electricity consumption for fans and pumps [15]. The electricity consumption is multiplied by a factor of three to convert site electricity to source fuel energy consumption.

6.1.2. Approximating cost functions. To compute approximations to the cost function $f(\cdot)$, we had to write a thermal building and daylighting simulation program, called BuildOpt [29, 30], because existing thermal building and daylighting simulation programs are built on models that do not satisfy the smoothness assumptions required to prove existence, uniqueness, and differentiability of $z(\cdot,1)$.

BuildOpt is a complex program that consists of two parts. The first part, which we will call the *simulation model generator*, parses a text input file with the detailed description of the building geometry, the building materials, and the expected occupancy behavior—which, for our problem, was $1,700$ lines long—and then generates a simulation model for the particular building, i.e., the functions $h(\cdot,\cdot,\cdot)$, $z_0(\cdot)$, and $\gamma(\cdot,\cdot,\cdot)$ of the DAE system (19). These functions are representations of various detailed models for the heat and daylighting transfer processes and for the building control systems. For example, the heat conduction in walls and ceilings are modeled using the Galerkin finite element method [11], and the transmittances of solar radiation and daylight through the windows are modeled using state-of-the-art optical calculations similar to those in commercial programs [12, 34]. There is also a detailed daylighting model that computes the available daylight at various locations in the building for different building and window configurations, and there are models for various control systems, such as the room lighting system and the heating and cooling system. The second part of BuildOpt, to which our simulation model generator was linked, is the commercial solver DASPK [6, 7].

The total size of BuildOpt is $38,000$ lines of C/C++ and Fortran code, of which $30,000$ lines (1.2 MB) of C/C++ code represent the simulation model generator and $8,000$ lines (0.3 MB) of Fortran code represent the commercial solver DASPK.

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

TABLE 1
Normalized computation times required to solve the building energy optimization problem with Algorithm 4.6. For each α, the last column shows the smallest Δ_k used in the search.

	$\zeta = 0$	$\zeta = 10^{-8}$	$\zeta = 10^{-6}$	$\zeta = 10^{-4}$	$\zeta = 10^{-2}$	Δ_{k^*}
$\alpha = 1/7$	0.27	0.27	0.27	0.28	0.55	1/2
$\alpha = 1/6$	0.33	0.33	0.33	0.31	0.61	1/4
$\alpha = 1/4$	0.35	0.35	0.35	0.31	0.74	1/4
$\alpha = 1/3$	0.55	0.55	0.55	0.60	1.21	1/8

We constructed the models in such a way that the functions $h(\cdot,\cdot,\cdot)$, $z_0(\cdot)$, and $\gamma(\cdot,\cdot,\cdot)$ are once Lipschitz continuously differentiable in all arguments, which has not been done before for thermal building and daylighting simulation programs. This required various smoothing techniques to replace conditional statements, which were in fact required to achieve convergence of the DASPK solver when the solver tolerance was tight.

In [31], BuildOpt was validated using the ANSI/ASHRAE Standard test procedure 140-2001 [2] for the thermal models and benchmark tests [18, 13] produced in the Task 21 of the International Energy Agency (IEA) Solar Heating & Cooling Program for the daylighting models. The validation results of BuildOpt show good agreement with the results of the other validated programs.

The computation time for one cost function evaluation was 24 sec for a solver tolerance of $\epsilon = 10^{-1}$, 2 min 22 sec for $\epsilon = 10^{-2}$, 5 min 42 sec for $\epsilon = 10^{-3}$, 16 min 30 sec for $\epsilon = 10^{-4}$, and 33 min 23 sec for $\epsilon = 10^{-5}$.

6.1.3. Optimization algorithm. We solved the optimization problem with adaptive precision cost function evaluations using the Hooke–Jeeves algorithm of the GenOpt$^{(R)}$ 2.0.0 optimization program [28] with the precision controlled as in GPS Algorithm model 4.6. For comparison, we also solved the problem using the Hooke–Jeeves optimization algorithm with fixed precision cost function evaluations and $\zeta = 0$. In the optimization with fixed precision cost function evaluations, we set $\epsilon_k = 10^{-5}$ for all $k \in \mathbb{N}$ and we allowed the mesh size to be decreased four times before the optimization stopped.

6.1.4. Precision control. Present day DAE solvers, such as DASPK, typically control the local error at each time step and do not even attempt to control the global error directly. We assumed that the global error of the approximate solutions $z^*(\epsilon, x, 1)$ is one order of magnitude greater than the local error. Hence, we set $\varphi(\epsilon) = 10\,\epsilon$. (Alternatively, we could have absorbed the factor 10 in the constant $K_{\mathbf{S}}$ in (2a).)

We defined $\rho\colon \mathbb{N} \to \mathbb{R}_+$ as $\rho(N) = 10^{-N}$ and increased precision four times. Thus, $\epsilon_k = \rho(1) = 10^{-1}$ for the first iterations, and $\epsilon_k = \rho(5) = 10^{-5}$ for the last iterations, which is equal to the precision used in the optimization with fixed precision cost function evaluations.

6.1.5. Numerical results. In Table 1, we show the values that we selected for the algorithm parameters $\alpha \in (0, 1)$ and $\zeta \geq 0$, the corresponding normalized computation times, and in the last column the smallest mesh size parameter Δ_{k^*}. A computation time of 1 corresponds to 5.5 days of computing, which was the time required to solve the optimization problem with the Hooke–Jeeves algorithm with fixed precision cost function evaluations and $\zeta = 0$.

Note that in Algorithm 4.6, the parameter $\alpha \in (0, 1)$ is only used to adjust the

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

mesh size parameter Δ_k so that $\varphi(\epsilon_k)^\alpha \geq \Delta_k^2$. Since $\varphi(\cdot)$ depends only on N, it is possible to compute for each $N \in \mathbb{N}$ the corresponding mesh size parameter. Such a computation shows that the sequence of mesh size parameters Δ_k, and hence the sequence of iterates x_k, are identical for all $\alpha \leq 1/7$, with $\alpha > 0$, and fixed ζ. Thus, a further reduction of α does not reduce the computation time.

For $\alpha \leq 1/4$, with $\zeta \in \{0, 10^{-8}, 10^{-6}, 10^{-4}\}$, our precision control algorithm reduces the computation time by a factor of three to four. For $\alpha = 1/7$ and $\zeta \leq 10^{-4}$, our precision control subprocedure reduced the computation time from about five days to one day, making the optimization fast enough to be applicable in building design processes. For our optimization problem, $\alpha = 1/3$ and $\zeta \geq 10^{-2}$ turn out to be too big, and imposing a sufficient decrease condition by setting $\zeta > 0$ does not reduce the computation time. All optimization runs converged to $x^* = (1, 1, 1, 0.19, 0.048)^T$ and reduced the energy consumption for lighting, cooling, and heating by 4.6 % or 9.4 kWh/(m^2 a).[1] The 4.6 % reduction is small but not representative for average savings, because in the literature [1, 32, 33] savings of 5% to 30% in energy consumption for lighting, cooling, and heating due to optimized building design have been reported. A reduction of 15%, which is more representative for average savings than the 4.6% that we achieved in our experiment, would correspond to a reduction in energy consumption for lighting, cooling, and heating of 30 kWh/(m^2 a). For an average current energy cost of \$0.10 per kWh, this corresponds to annual savings of \$3 per square meter floor area, or to annual savings of \$30,000 for a large, 10,000 m^2 office building. As large buildings are often designed using energy simulations, and hence a computer simulation model exists for those buildings, the additional effort to do an optimization is only a few man hours. Thus, the return-of-investment is achieved within the first year of the building operation time.

We will now describe how the optimization runs with fixed and adaptive precision cost function evaluations, with $\zeta = 10^{-4}$ and $\alpha = 1/6$, converged to a minimum. Let the normalized distance of the kth iterate $x_k \in \mathbb{R}^n$ to the minimizer $x^* \in \arg\min_{x \in \mathbf{X}} f(x)$, produced by the optimization algorithm, be defined as

$$d(x_k) \triangleq \frac{\|x_k - x^*\|}{\|x_0 - x^*\|}, \tag{21}$$

where $x_0 \in \mathbb{R}^n$ is the initial iterate. Figure 1 shows the cost function value and the distance to the minimizer as a function of the computation time. Below the axis we show when precision was increased. The different precision values are indicated by ϵ_m, $m \in \{0, 1, 2, 3, 4\}$, where $\epsilon_m = 10^{-(m+1)}$. The abscissa in Figure 1 shows the normalized CPU time rather than the number of cost function evaluations, because evaluating $f^*(10^{-1}, \cdot)$ was eighty times faster than evaluating $f^*(10^{-5}, \cdot)$ in our experiments (see also [29, 30]). In the left graph, we can see that even for such coarse a precision as $\epsilon = 10^{-1}$, the approximating cost function $f^*(10^{-1}, \cdot)$ allowed a substantial decrease in cost during the first 0.2% of the computation time.

6.2. Optimization problem with cost function defined on the solutions of a nonlinear system of equations. We will now present the computational performance of our precision control algorithms in minimizing a cost function that is defined on the approximate solutions of a nonlinear system of equations with 373 unknowns.

[1] The unit kWh/(m^2 a) is kilowatt hours per square meter floor area per year.

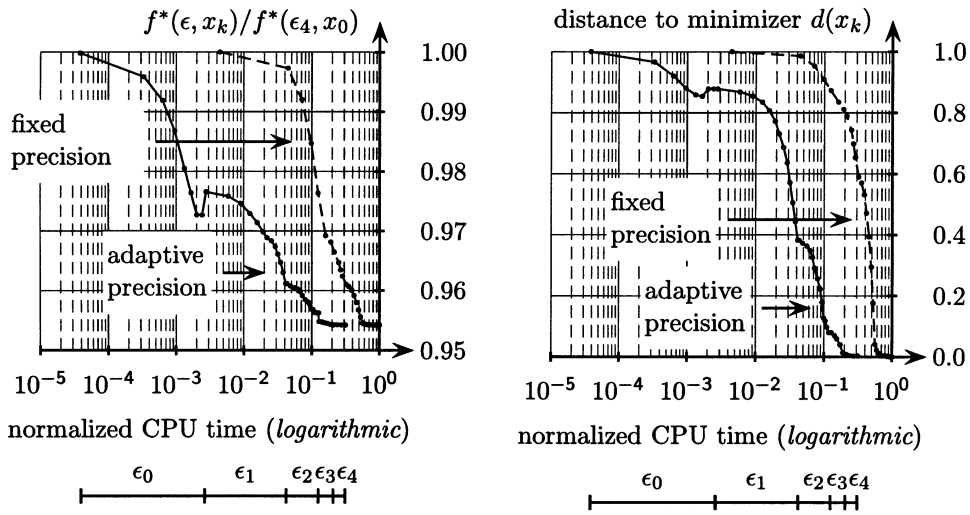

FIG. 1. *Normalized cost function value (left graph) and distance to the minimizer (right graph) as a function of the normalized CPU time in logarithmic scale. Below the graphs we show the intervals for which the precision parameter ϵ has been kept constant. For the adaptive precision optimization, we used $\zeta = 10^{-4}$ and $\alpha = 1/6$. For better display of the early iterations, the time axis is in logarithmic scale.*

The objective is to fit four parameters of a detailed air-to-water cooling coil computer simulation model in such a way that the difference between simulated and measured coil air outlet temperature is minimized for a prescribed number of measurement points. The measurement data are the air and water inlet temperature, the air humidity ratio, the air and water mass flow, and the valve position of the throttle valve in the water circuit. There are 401 measurement data, equally spaced in time.

The simulation model [35] consists of a coupled system of nonlinear equations that is solved for 373 variables using Newton iterations. The model is static and was simulated in SPARK 1.0.3 [19]. For the range of measurement data, all model equations are once continuously differentiable, and the Jacobian matrix is nonsingular in a neighborhood of the solution. Therefore, it follows from the implicit function theorem [23] that the exact solution, and hence the cost function, is once continuously differentiable.

The design parameters are the air and water side heat transfer coefficients and two parameters that define the valve characteristics. We controlled two precision parameters: the precision parameter for the Newton solver and the number of data points that were used in the data fit.

6.2.1. Exact cost function. We defined the exact cost function as follows. Let $\tau \triangleq [0,\ 1]$ denote the normalized time interval over which the measurement took place. Let $T_m : [0,\ 1] \to \mathbb{R}$ be the linear interpolation of the measured coil air outlet temperatures. For $x \in \mathbb{R}^n$ and $t \in \tau$, let $T_s(x,t) \in \mathbb{R}$ denote the exact solution of the system of equations that defines the coil air outlet temperature, obtained by using linearly interpolated measurement data. Then, for

$$e(x,t) \triangleq (T_m(t) - T_s(x,t))^2, \tag{22}$$

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

we defined the exact cost function

$$f(x) \triangleq \int_0^1 e(x,t)\,dt. \tag{23}$$

6.2.2. Approximating cost functions. The integral (23) cannot be evaluated because $T_s(x,t)$ can only be numerically approximated by an approximate solution $T_s^*(\epsilon^1, x, t) \in \mathbb{R}$ with precision parameter $\epsilon^1 \in \mathbb{R}_+$, and the integral can only be approximated by a quadrature formula. Thus, in computing the approximating cost functions, we have two sets of approximations.

We approximated $f(\cdot)$ as follows. For some $\epsilon_0 \in \mathbb{R}_+^2$, with $\epsilon_0^2 \leq 1/2$, let $\epsilon^1 \in (0, \epsilon_0^1]$ denote the precision parameter of the Newton solver, and let $\epsilon^2 \in (0, \epsilon_0^2]$ denote the time interval for the quadrature formula. For $t \in \tau$, we approximated (22) by

$$e^*(\epsilon^1, x, t) \triangleq (T_m(t) - T_s^*(\epsilon^1, x, t))^2, \tag{24a}$$

using Newton iterations. The Newton solver in the SPARK program is set up in such a way that for any compact set $\mathbf{S} \subset \mathbf{X}$, there exists an $\epsilon_{\mathbf{S}} \in \mathbb{R}_+$ and a $K'_{\mathbf{S}} \in (0, \infty)$ such that for all $x \in \mathbf{S}$, for all $t \in [0, 1]$, and for all $\epsilon \in \mathbb{R}_+$, with $\epsilon \leq \epsilon_{\mathbf{S}}$,

$$|e^*(\epsilon, x, t) - e(x,t)| \leq K'_{\mathbf{S}}\,\epsilon. \tag{24b}$$

We approximated the integral (23) by

$$f^*(\epsilon, x) \triangleq \sum_{i=0}^{N(\epsilon^2)-1} \frac{e^*(\epsilon^1, x, i/N(\epsilon^2)) + e^*(\epsilon^1, x, (i+1)/N(\epsilon^2))}{2\,N(\epsilon^2)}, \tag{25}$$

where $N(\epsilon^2) \triangleq \lfloor 1/\epsilon^2 \rfloor$.

It can be shown that for any compact set $\mathbf{S} \subset \mathbb{R}^n$, there exist a $K_{\mathbf{S}} \in (0, \infty)$ and an $\epsilon_{\mathbf{S}} \in \mathbb{R}_+^2$ such that

$$|f^*(\epsilon, x) - f(x)| \leq K_{\mathbf{S}}\,\|\epsilon\|, \tag{26}$$

for all $x \in \mathbf{S}$ and for all $\epsilon \in \mathbb{R}_+^2$, with $\epsilon \leq \epsilon_{\mathbf{S}}$. Therefore, $\varphi(\cdot)$ in Assumption 3.1 is $\varphi(\epsilon) = \|\epsilon\|$.

6.2.3. Optimization algorithm. Numerical experiments showed that $f(\cdot)$ has several local minima, and some local minima have a cost function value that is three times larger than the best found solution. Therefore, we used the multistart Hooke–Jeeves optimization algorithm from the GenOpt$^{(R)}$ 2.0.0 optimization program [28] with four randomly selected initial iterates. We controlled the precision of the approximating cost functions using the precision control algorithm from GPS Algorithm model 4.4. For comparison, we also solved the problem with the multistart Hooke–Jeeves algorithm with fixed precision $\epsilon_k = (10^{-10}, 1/400)^T$ for all $k \in \mathbb{N}$, with $\zeta = 0$ and three step reductions.

6.2.4. Precision control. To control $\epsilon_k \in \mathbb{R}_+^2$, with $\epsilon_k \leq \epsilon_0$, as a function of the mesh size factor $\Delta_k \in \mathbb{Q}_+$, we defined $\rho\colon \mathbb{R}_+ \to \mathbb{R}_+^2$ as

$$\rho^i(\Delta) \triangleq \epsilon^i_{\min} \left(\frac{\Delta}{\Delta_{min}}\right)^{\alpha^i}, \qquad \alpha^i > 1, \qquad i \in \{1, 2\}, \tag{27}$$

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

TABLE 2
Initial precisions ϵ_0 used for approximating the cost functions, corresponding α, normalized computation time, and best obtained local minima for all optimizations.

ϵ_0^1	ϵ_0^2	α^1	α^2	CPU time	$f^*(\epsilon, x^*)$
10^{-10}	1/400	0	0	1	0.0236
10^{-1}	0.025	9.97	1.11	0.13	0.0242
10^{-1}	0.05	9.97	1.44	0.15	0.0225
10^{-1}	0.1	9.97	1.77	0.72	0.0216
10^{-1}	0.5	9.97	2.55	0.34	0.0217
10^{-1}	1	9.97	2.88	0.51	0.0221
10^{-2}	0.025	8.86	1.11	0.12	0.0242
10^{-2}	0.05	8.86	1.44	0.15	0.0225
10^{-2}	0.1	8.86	1.77	0.69	0.0216
10^{-2}	0.5	8.86	2.55	0.18	0.0217
10^{-2}	1	8.86	2.88	0.54	0.0221
10^{-3}	0.025	7.75	1.11	0.13	0.0242
10^{-3}	0.05	7.75	1.44	0.15	0.0225
10^{-3}	0.1	7.75	1.77	0.69	0.0216
10^{-3}	0.5	7.75	2.55	0.18	0.0217
10^{-3}	1	7.75	2.88	0.53	0.0221
average for adaptive precision				0.35	

where $\Delta_{min} \triangleq \min_{k\in\mathbb{N}}\{\Delta_k\} = 1/8$ is the smallest mesh size parameter, and $\epsilon_{min} = (10^{-10}, 1/400)^T$ is the precision parameter for the last iterations. Since $\alpha^i > 1$ for $i \in \{1, 2\}$, we have $\varphi(\rho(\Delta))/\Delta \to 0$, as $\Delta \to 0$.

To determine $\alpha > 1$, we selected different initial precisions $\epsilon_0 \in \mathbb{R}^2_+$ and then computed α by solving (27) with $\Delta = \Delta_0 = 1$ and $\rho^i(1) = \epsilon_0^i$ for $i \in \{1, 2\}$. In particular, we set

$$\alpha^i = \frac{\log\left(\epsilon_0^i/\epsilon_{min}^i\right)}{\log\left(\Delta_0/\Delta_{min}\right)} = \frac{\log\left(\epsilon_0^i/\epsilon_{min}^i\right)}{\log 8}, \qquad i \in \{1, 2\}. \tag{28}$$

Thus, for $\alpha^i > 1$, we need $\epsilon_0^i > 8\,\epsilon_{min}^i$ for $i \in \{1, 2\}$.

6.2.5. Numerical results. Table 2 shows the different settings for ϵ_0, the corresponding α, the normalized computation time, and the cost function values for the best obtained local minima. A normalized computation time of one corresponds to 45 minutes of computation time, which was the time required to solve the optimization problem with fixed precision cost function evaluations.

All optimization runs obtained a similar reduction in cost. On average, our precision control scheme reduced the computation time by a factor of three.

We will now describe how the optimizations with fixed and adaptive precision cost function evaluations, with $\zeta = 0$ and $\alpha = (9.97,\ 1.11)^T$, converged to a local minimum. Let the normalized distance to the best local minimizer of the optimization that used fixed precision cost function evaluations be defined as in (21). Figure 2 shows, for the optimizations with adaptive and fixed precision cost function evaluations, the cost function value and the normalized distance to the minimizer as a function of the computation time. For the initial iterate and the precision control parameter α used in the optimizations shown in Figure 2, both algorithms converged to the same local minimum. Below the axis we show when precision was increased (the different precisions are indicated by ϵ_m, $m \in \{0, 1, 2, 3\}$). For this example, the precision control algorithm set $\epsilon_0 = (10^{-1}, 0.025)^T$, $\epsilon_1 = (10^{-4}, 0.012)^T$, $\epsilon_2 = (10^{-7}, 0.0054)^T$, and $\epsilon_3 = (10^{-10}, 0.0025)^T$. After 3% of the computation time that was required to

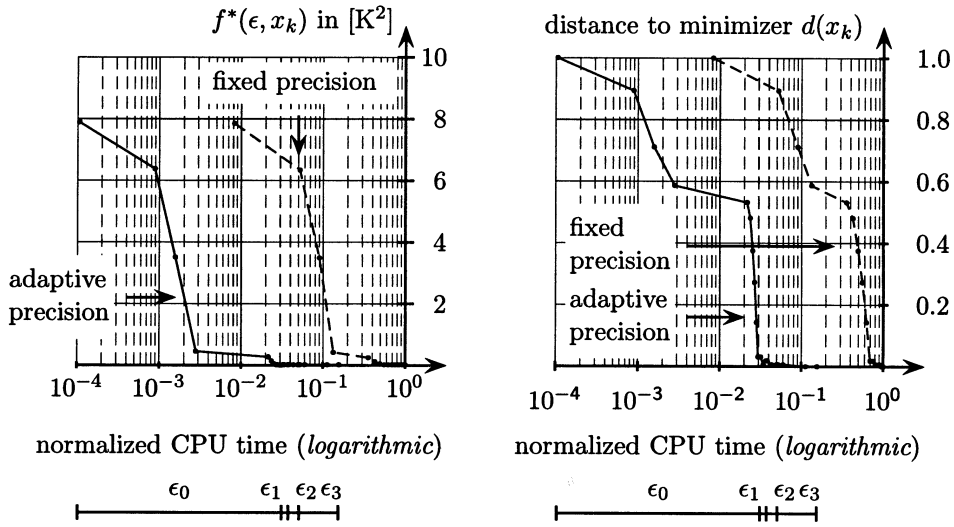

FIG. 2. *Cost function value (left graph) and distance to the minimizer (right graph) as a function of the normalized computation time in logarithmic scale. Below the graphs we show the intervals for which the precision parameter ϵ has been kept constant. For the adaptive precision optimization, we used $\zeta = 0$ and $\alpha = (9.97, 1.11)^T$. For better display of the early iterations, the time axis is in logarithmic scale.*

solve the fixed precision optimization problem, the cost function values and the iterates of the adaptive precision optimization problem were already close to the minimum.

7. Conclusion. We have extended the family of GPS algorithms to a form that converges to a stationary point of a smooth cost function that cannot be evaluated exactly, but that can be approximated by a family of possibly discontinuous functions $\{f^*(\epsilon, \cdot)\}_{\epsilon \in \mathbb{R}^q_+}$. An important feature of our algorithms is that they use low-cost, coarse precision approximations to the cost function when far from a solution, with the precision progressively increased as a solution is approached. We have shown by numerical experiments that our precision control algorithms lead to considerable time savings over using high precision approximations to the cost function in all iterations.

Acknowledgment. We thank the referees for their comments that simplified the presentation of this work.

REFERENCES

[1] M. S. Al-Homoud, *Optimum thermal design of office buildings*, Intern. J. Energy Res., 21 (1997), pp. 941–957.

[2] ASHRAE, *ANSI/ASHRAE Standard* 140-2001, Standard method of test for the evaluation of building energy analysis computer programs, 2001.

[3] C. Audet and J. E. Dennis, Jr., *Analysis of generalized pattern searches*, SIAM J. Optim., 13 (2003), pp. 889–903.

[4] A. J. Booker, J. E. Dennis, Jr., P. D. Frank, D. B. Serafini, V. Torczon, and M. W. Trosset, *A rigorous framework for optimization of expensive functions by surrogates*, Struct. Optim., 17 (1999), pp. 1–13.

[5] K. E. Brenan, S. L. Campbell, and L. R. Petzold, *Numerical Solution of Initial-Value Problems in Differential-Algebraic Equations*, North–Holland, Amsterdam, 1989.

[6] P. N. Brown, A. C. Hindmarsh, and L. R. Petzold, *Using Krylov methods in the solution of large-scale differential-algebraic systems*, SIAM J. Sci. Comput., 15 (1994), pp. 1467–1488.

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

[7] P. N. Brown, A. C. Hindmarsh, and L. R. Petzold, *Consistent initial condition calculation for differential-algebraic systems*, SIAM J. Sci. Comput., 19 (1998), pp. 1495–1512.

[8] F. H. Clarke, *Optimization and Nonsmooth Analysis*, Classics Appl. Math. 5, SIAM, Philadelphia, 1990.

[9] J. E. Dennis, Jr., and V. Torczon, *Direct search methods on parallel machines*, SIAM J. Optim., 1 (1991), pp. 448–474.

[10] J. E. Dennis, Jr. and V. Torczon, *Managing approximation models in optimization*, in Multidisciplinary Design Optimization: State of the Art, ICASE/NASA Langley Workshop on Multidisciplinary Optimization, N. M. Alexandrov and M. Y. Hussaini, eds., SIAM, Philadelphia, 1997, pp. 330–347.

[11] L. C. Evans, *Partial Differential Equations*, AMS, Providence, RI, 1998.

[12] E. U. Finlayson, D. K. Arasteh, C. Huizenga, M. D. Rubin, and M. S. Reilly, *WINDOW* 4.0: *Documentation of Calculation Procedures*, Tech. report LBL-33943, Lawrence Berkeley National Laboratory, Berkeley, CA, 1993.

[13] M. Fontoynont, P. Laforgue, R. Mitanchey, M. Aizlewood, J. Butt, W. Carroll, R. Hitchcock, H. Erhorn, J. De Boer, M. Dirksmöller, L. Michel, B. Paule, J. L. Scartezzini, M. Bodart, and G. Roy, *IEA SHC Task* 21*/ECBCS Annex* 29*, Daylight in Buildings, Subtask C1: Validation of Daylighting Simulation Programmes*, Tech. report T21/C1-/FRA/99-11, International Energy Agency, 1999.

[14] R. Hooke and T. A. Jeeves, *"Direct search" solution of numerical and statistical problems*, J. Assoc. Comput. Machinery, 8 (1961), pp. 212–229.

[15] J. Huang and E. Franconi, *Commercial Heating and Cooling Loads Component Analysis*, Tech. report LBL-37208, Lawrence Berkeley National Laboratory, EETD, 1999.

[16] C. T. Kelley, *Iterative Methods for Optimization*, Frontiers Appl. Math. 18, SIAM, Philadelphia, 1999.

[17] T. G. Kolda, R. M. Lewis, and V. Torczon, *Optimization by direct search: New perspectives on some classical and modern methods*, SIAM Rev., 45 (2003), pp. 385–482.

[18] P. Laforgue, *IEA SHC Task* 21*/ECBCS Annex* 29*, Daylight in Buildings, Subtask C1: Draft Report of Genelux Simulations and Other Software Results*, Tech. report T21/C1/97-10, International Energy Agency, 1997.

[19] L. Berkeley National Laboratory and A. Sowell Associates Inc., *SPARK, Reference Manual*, Berkeley, CA, 2003.

[20] R. M. Lewis and V. Torczon, *Pattern search algorithms for bound constrained minimization*, SIAM J. Optim., 9 (1999), pp. 1082–1099.

[21] R. M. Lewis and V. Torczon, *Pattern search methods for linearly constrained minimization*, SIAM J. Optim., 10 (2000), pp. 917–941.

[22] E. Polak, *Computational Methods in Optimization; A Unified Approach*, Math. in Sci. and Engrg 77, Academic Press, New York, 1971.

[23] E. Polak, *Optimization, Algorithms and Consistent Approximations*, Appl. Math. Sci. 124, Springer, New York, 1997.

[24] E. Polak and M. Wetter, *Generalized Pattern Search Algorithms with Adaptive Precision Function Evaluations*, Tech. report LBNL-52629, Lawrence Berkeley National Laboratory, Berkeley, CA, 2003.

[25] D. B. Serafini, *A Framework for Managing Models in Nonlinear Optimization of Computationally Expensive Functions*, Ph.D thesis, Rice University, Houston, TX, 1998.

[26] V. Torczon, *On the convergence of pattern search algorithms*, SIAM J. Optim., 7 (1997), pp. 1–25.

[27] V. Torczon and M. W. Trosset, *Using approximations to accelerate engineering design optimization*, in Proceedings of the 7th Annual AIAA/USAF/NASA/ISSMO Symposium on Multidisciplinary Analysis and Optimization, AIAA Paper 98-4800, St. Louis, MO, 1998.

[28] M. Wetter, *GenOpt, Generic Optimization Program, User Manual, Version* 2.0.0, Tech. report LBNL-54199, Lawrence Berkeley National Laboratory, Berkeley, CA, 2004.

[29] M. Wetter, *Simulation-Based Building Energy Optimization*, Ph.D thesis, University of California at Berkeley, Berkeley, CA, 2004.

[30] M. Wetter, *BuildOpt – A new building energy simulation program that is built on smooth models*, Building and Environment, 40 (2005), pp. 1085–1092.

[31] M. Wetter, E. Polak, and V. P. Carey, *BuildOpt* 1.0.1 *Validation*, Tech. report LBNL-54658, Lawrence Berkeley National Laboratory, Berkeley, CA, 2004.

[32] M. Wetter and J. Wright, *Comparison of a generalized pattern search and a genetic algorithm optimization method*, in Proceedings of the 8th Annual IBPSA Conference, G. Augenbroe and J. Hensen, eds., vol. III, Eindhoven, NL, 2003, pp. 1401–1408.

[33] M. Wetter and J. Wright, *A comparison of deterministic and probabilistic optimization algorithms for nonsmooth simulation-based optimization*, Building and Environment, 39 (2004), pp. 989–999.

[34] F. Winkelmann, *Modeling windows in EnergyPlus*, in Proceedings of the 7th Annual IBPSA Conference, Vol. 1, R. Lamberts, C. O. R. Negrão, and J. Hensen, eds., Rio de Janeiro, Brazil, 2001, pp. 457–464.

[35] P. Xu and P. Haves, *Library of Component Reference Models for Fault Detection (AHU and Chiller)*, Tech. report LBNL for the California Energy Commission, Lawrence Berkeley National Laboratory, Berkeley, CA, 2001.

Downloaded 09/28/24 to 183.169.75.170 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy